

Automated Radar Tracking & Interception System

Academic EEE Proof-of-Concept Project Blueprint

1. Project Overview

This document outlines the architecture, budget, and implementation plan for a theoretical anti-drone tracking and interception system. Designed strictly for academic engineering demonstration, this project utilizes low-cost FMCW radar, a high-efficiency Mini PC, and a servo-driven laser pointer to simulate target acquisition and interception.

The system operates autonomously 24/7, tracking moving targets (simulated drones) in a localized environment (5–10 meters) and calculating predictive trajectories using basic Machine Learning and filtering algorithms.

2. Component List & Budget Allocation

The project is designed to strictly adhere to a 20,000 BDT limit, leveraging local availability and utilizing an existing ATX power supply for power distribution.

Component	Description & Purpose	Est. Cost (BDT)
Qbits Lania Mini F	Intel N95, 8GB RAM. Acts as the central command node, running Python targeting logic.	15,999
HLK-LD2450 24GHz Radar	FMCW radar module. Outputs X/Y coordinates and velocity of moving targets.	1,600
ESP32 or Arduino Uno	Microcontroller to act as a bridge between the PC's USB and the servo PWM signals.	650
Pan/Tilt Bracket + Servos	Dual-axis bracket with 2x MG996R servos for aiming the interceptor.	1,100
Laser Pointer Module	5V laser module to simulate the theoretical "interceptor" strike.	150
Misc. Electronics	DC Barrel Jack (5.5x2.1mm), jumper wires, USB-to-TTL module (if needed).	300

Total Estimated Budget:

~19,799
BDT

⚠ POWER MODIFICATION WARNING: You will be powering the Qbits Mini PC (12V) directly from an ATX power supply. You MUST ensure proper polarity. The DC barrel jack is center-positive. Wiring the 12V (Yellow) and GND (Black) backward will permanently destroy the Mini PC.

3. System Architecture & Wiring Diagram

The architecture relies on serial communication. The radar sends coordinate data to the PC; the PC calculates the required angles, and sends commands to the microcontroller to move the servos.

```
[ATX PSU (Yellow 12V, Black GND)] ---> (DC Barrel Jack) ---> [Qbits Mini PC]
[ATX PSU (Red 5V, Black GND)] -----> [ESP32/Arduino & Servo Motors]

[HLK-LD2450 Radar] --(UART TX/RX)--> [USB-TTL] --(USB)--> [Qbits Mini PC]
[Qbits Mini PC] --(USB Serial)--> [ESP32/Arduino] --(PWM)--> [Pan/Tilt Servos]
```

Pin Connections:

- **Radar to PC:** LD2450 TX to USB-TTL RX, LD2450 RX to USB-TTL TX, 5V to VCC, GND to GND.
- **Microcontroller to Servos:** Pan Servo PWM to Digital Pin 9, Tilt Servo PWM to Digital Pin 10. Power servos directly from the ATX 5V line, NOT the microcontroller's 5V pin.

4. Targeting Mathematics

The LD2450 outputs Cartesian coordinates (x, y) in millimeters. To aim the laser, these must be converted to polar coordinates (angles) for the Pan and Tilt servos. The base calculation for the Pan angle (θ) relative to the radar is:

$$\theta = \arctan(y / x) \times (180 / \pi)$$

For predictive targeting, a basic filtering algorithm (like an Alpha-Beta filter or a simplified Kalman Filter) can be applied to the velocity data to aim ahead of the target.

5. Software Implementation

Step 1: Python Processing Logic (Runs on Qbits Mini PC)

This script reads the radar data, converts coordinates to angles, and sends the command to the Arduino.

```

import serial
import math
import time

# Configure Serial Ports (Check COM ports in Windows Device Manager)
RADAR_PORT = 'COM3' # Port for LD2450 via USB-TTL
ARDUINO_PORT = 'COM4' # Port for Arduino/ESP32
BAUD_RATE = 256000 # LD2450 default baud rate

try:
    radar_serial = serial.Serial(RADAR_PORT, BAUD_RATE, timeout=0.1)
    arduino_serial = serial.Serial(ARDUINO_PORT, 9600, timeout=1)
    print("System Armed. Listening for radar signatures...")
except Exception as e:
    print(f"Hardware connection failed: {e}")
    exit()

def calculate_angles(x, y):
    # Avoid division by zero
    if x == 0:
        return 90

    # Calculate Pan angle (Theta) in degrees
    angle_rad = math.atan2(y, x)
    pan_angle = math.degrees(angle_rad)

    # Map to 0-180 degree servo range
    pan_angle = int(max(0, min(180, pan_angle)))
    return pan_angle

while True:
    # Read basic data frame from radar (simplified for demonstration)
    if radar_serial.in_waiting > 0:
        raw_data = radar_serial.readline().decode('utf-8', errors='ignore').strip()

        # Example parsing (Depends on exact LD2450 firmware output format)
        # Assuming format: "X: 1200, Y: 3000, V: 5"
        try:
            parts = raw_data.split(',')
            x_val = float(parts[0].split(':')[1].strip())
            y_val = float(parts[1].split(':')[1].strip())

            pan_target = calculate_angles(x_val, y_val)
            tilt_target = 90 # Fixed elevation for 2D demo

            # Send to Arduino: Format "P(angle)T(angle)"
            command = f"P{pan_target}T{tilt_target}"

            arduino_serial.write(command.encode('utf-8'))
            print(f"Target Acquired at X:{x_val}, Y:{y_val} -> Aiming Pan: {pan_target}")

        except (IndexError, ValueError):

```

```
pass # Ignore malformed serial data
```

```
time.sleep(0.05) # 50ms loop rate for real-time tracking
```

Step 2: Arduino Servo Control Code (Runs on Microcontroller)

Upload this via the Arduino IDE to handle the mechanical interception movement.

```
#include <Servo.h>

Servo panServo;
Servo tiltServo;

int panPin = 9;
int tiltPin = 10;

void setup() {
  Serial.begin(9600);
  panServo.attach(panPin);
  tiltServo.attach(tiltPin);

  // Initialize to center positions
  panServo.write(90);
  tiltServo.write(90);
}

void loop() {
  if (Serial.available() > 0) {
    String data = Serial.readStringUntil('
');

    // Parse incoming data, e.g., "P120T45"
    int pIndex = data.indexOf('P');
    int tIndex = data.indexOf('T');

    if (pIndex != -1 && tIndex != -1) {
      String panStr = data.substring(pIndex + 1, tIndex);
      String tiltStr = data.substring(tIndex + 1);

      int panAngle = panStr.toInt();
      int tiltAngle = tiltStr.toInt();

      // Execute tracking movement
      panServo.write(panAngle);
      tiltServo.write(tiltAngle);
    }
  }
}
```

| 6. Testing and Calibration Procedure

1. **Power Test:** Boot the PC using the custom ATX 12V cable BEFORE connecting USB devices. Ensure it boots cleanly into Windows/Linux.
2. **Radar Calibration:** Place the radar 1 meter off the ground. Walk in front of it. Monitor the Python console to ensure X/Y coordinates change accurately.
3. **Interception Test:** Mount the laser to the pan/tilt bracket. Walk across the room. The servos should smoothly track your movement, keeping the laser dot on your center mass.

End of Document.